STRIPEDHYENA 2 · NATIVE PYTORCH · AWS TRAINIUM

Evo 2 on Trainium

Arc Institute's 7B DNA language model running on `torch.device("neuron")` in plain PyTorch — no vortex runtime, no TransformerEngine, no flash-attn, no CUDA. The same package runs unchanged on CPU. Gated against saved H200 activations, with generic LoRA fine-tuning and a data-parallel driver for every core on the instance.

`CUDA→CPU` 0 diverging blocks | `CUDA→Trainium` 0 diverging blocks | `seq 8192` 0 diverging blocks

`grad + 3 steps` 74/74 tensors | `host fallbacks` none | `64 cores` 147,292 tok/s · 96.7%

Install and weights

Assumes the AWS native-PyTorch beta is already installed — that is where `torch`, `torch-neuronx`, `neuronx-cc` and `nki` come from. The package deliberately does not pin them; installing them from PyPI would shadow the editable beta install.

```
# the beta env, then this package's two dependencies
conda activate torch-neuron
pip install -r requirements.txt
export PYTHONPATH=$PWD/src

# works with no device and no weights: 100 unit tests, ~20 s
make test
make smoke      # every example on CPU, random weights
```

Weights are Arc's and are not redistributed. They download on first use. A read-out head only needs blocks 0..10, so pruning the checkpoint once cuts it from 13 GB to 4.5 GB — worth doing before a multi-rank job, where every rank reads it.

```
huggingface-cli download arcinstitute/evo2_7b_base evo2_7b_base.pt

python scripts/prune_checkpoint.py evo2_7b_base.pt evo2_7b_blocks0_10.pt --max-blocks 11
export EVO2_CHECKPOINT=$PWD/evo2_7b_blocks0_10.pt
```

Embeddings in six lines

```
import evo2_finetune.neuron_env as env; env.configure()   # BEFORE torch
import torch
from evo2_finetune.backbone import Evo2Backbone
from evo2_neuron import ByteTokenizer

backbone = Evo2Backbone("evo2-7b").to(env.resolve_device("neuron"))
ids, mask = ByteTokenizer().encode_batch(["ACGTACGTACGT"], max_length=8192)
hidden = backbone(ids.to("neuron"))          # (1, 8192, 4096)
```

ORDER MATTERS

`configure()` must run before anything imports torch. `import torch` auto-loads `torch_neuronx`, which reads the four compiler cache directories at that moment; setting them afterwards has no effect and the run silently uses `/tmp`. It raises rather than letting you find out later.

Parity gates

The reference is an NVIDIA H200 running evo2 0.1.0 / vortex 0.0.2, bit-reproducible across two physical GPUs. Its activations ship in the box, so the gates are runnable rather than quotable.

AXIS	WHAT IT PROVES	HIDDEN0	RESULT
CUDA → CPU, seq 200	the port reproduces vortex + TransformerEngine	1.48e-02	PASS
CUDA → Trainium, seq 200	end to end, reference to device	3.53e-02	PASS
CUDA → CPU, seq 8192	the production context	1.89e-02	PASS
CPU ↔ Trainium, forward fp32	the device lowering is faithful	8.95e-05	PASS
CPU ↔ Trainium, backward	gradients agree, 74 tensors	—	0 real
CPU ↔ Trainium, 3 AdamW steps	on-device training agrees	~6e-08	0 real

```
make health     # no orphaned process is holding a core
make parity-cpu # the CUDA axis only — no device needed
make parity     # all three gates, CPU and Neuron
make fallbacks  # per-stage, per-dtype host-fallback census
```

The gate is the residual stream, not the read-out

`hidden0` is a **0.42% correction** to the residual stream by norm: it reports *that* something diverged and never *where*. And the metric on the stream is the increment `delta_k = block_out.k - block_out.{k-1}`, not the cumulative tensor — block 3 contributes 1.3% of `block_out.3`, so a

completely broken attention block reads ~1.3e-02 cumulatively, inside the bf16 noise floor.

A block is flagged only if it fails **both** magnitude (`relerr > 3e-3`) and direction (`1-cos > 1e-2`). Both floors are calibrated against controls, not chosen.

> **WHY MAGNITUDE ALONE SAYS LITTLE AT BF16**
>
> Flipping `requires_grad` on the backbone moves `hidden0` by **1.244e-02** — same weights, same input, same process, only the autograd metadata differs, which is enough for PyTorch's CPU path to pick a different GEMM. The entire CUDA→CPU disagreement is 1.48e-02. At this precision the magnitude is mostly reduction order; direction is the half that discriminates.

LoRA fine-tuning

The backbone is frozen; only the adapters and the read-out head train — 74 tensors, 5.2 M of 2.23 B parameters at r=8. Six objectives over four heads, chosen by the granularity of the label you have.

OBJECTIVE	HEAD	LABEL	OUTPUT
`next-token-ce`	tied LM head	none — the sequence itself	`(B, L, 512)`
`sequence-regression`	MLPReadoutHead · ConvReadoutHead	one float vector per sequence	`(B, out)`
`sequence-classification`	MLPReadoutHead · ConvReadoutHead	one class per sequence	`(B, out)`
`window-regression`	WindowTrackHead	one value per 256 bp bin	`(B, L/256, out)`
`per-position-classification`	PerPositionHead	one class per nucleotide	`(B, L, out)`
`per-position-regression`	PerPositionHead	one value per nucleotide	`(B, L, out)`

```
from evo2_finetune import build_model

model = build_model(
    objective="window-regression",      # one prediction per 256 bp bin
    alias="evo2-7b", out_dim=1, device="neuron", dtype=torch.bfloat16,
    head_kwargs={"window": 256}, gradient_checkpointing=True,
)
out = model(input_ids=ids, attention_mask=mask, labels=y, weights=w)
out["loss"].backward()
```

Four runnable examples ship with the repo. Each runs on CPU in seconds with `--random-backbone` , and on Trainium with real weights.

```
python examples/01_embeddings.py --device cpu --random-backbone --max-blocks 3
python examples/02_lora_lm_finetune.py --fasta genome.fa --device neuron
python examples/03_window_regression.py --device neuron
python examples/04_per_nucleotide_classification.py --device neuron
```

Labelled data is optional: `SyntheticDNADataset` plants motifs in seeded random sequence and derives sequence, window and per-base labels from them, so every example runs offline. The signal is learnable on purpose — a flat loss curve there means the harness is wrong, not that the task is hard.

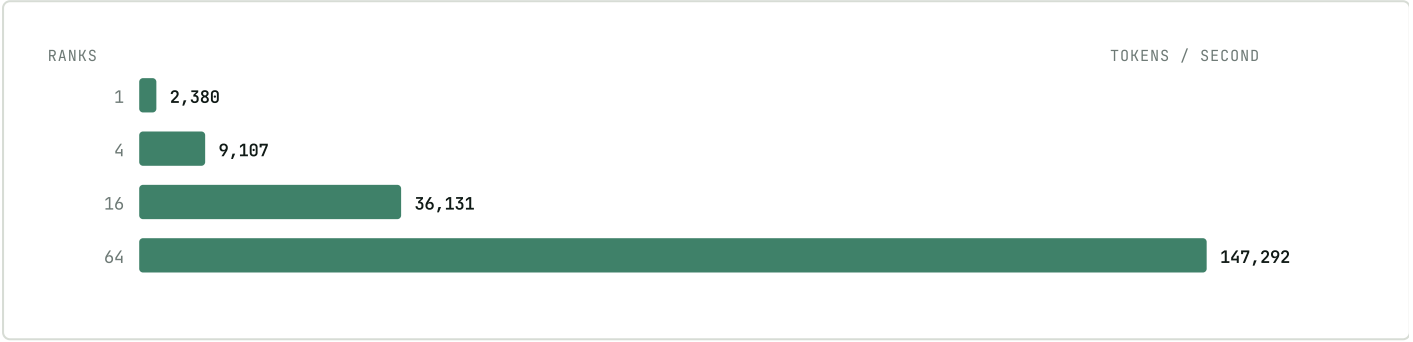
Training across every NeuronCore

Warm the compiler cache with **one** rank first — concurrent cold compiles have been enough to kill the compile service. The cold compile of eleven 7B blocks at 8192 is a one-off **932 s**; every warm step after it is **3.44 s**.

```
# warm the NEFF cache, one rank
NEURON_RT_VISIBLE_CORES=0 python -u scripts/train_lora_ddp.py --steps 12

# then all 64 logical cores of a trn2.48xlarge
NEURON_RT_VISIBLE_CORES=0-63 torchrun --nproc_per_node=64 \
    scripts/train_lora_ddp.py --objective window-regression --seq-len 8192 \
    --steps 200 --save-to runs/lora64 --diag-dir runs/lora64/diag
```

RANKS	COLD COMPILE	WARM STEP	TOKENS/S	SPEEDUP	EFFICIENCY
1	932 s	3.442 s	2,380	1.00×	100%
4	853 s	3.598 s	9,107	3.83×	96%
16	63 s	3.628 s	36,131	15.18×	95%
64	81 s	3.560 s	147,292	61.88×	97%



61.9× on 64 logical NeuronCores at 96.7% efficiency, 147,292 tokens/s at 8192 context. The step time moves 3.4% from one rank to sixty-four — 3.4418 s to 3.5595 s for a 64× wider batch — which is what data-parallel scaling looks like when the collective is small: the gradient payload is 10.3 MB in bf16 and fits in one bucket. Rank spread stays under 0.5%

throughout, so these are throughput numbers and not straggler numbers.

The two large cold compiles are the first non-DDP graph and the first DDP graph. Once the DDP graph is cached, 16 and 64 ranks start in 63 s and 81 s — which is the whole argument for warming with one multi-rank job before launching a big one.

Defaults are the production configuration: bf16, per-block compile, activation checkpointing, `nki-scan`, batch 1 per rank. Batch > 1 at 8192 compiles and then fails to *load* on a 9.5 MB allocation — that is fragmentation, not exhaustion, which is why scale comes from ranks and not from batch.

What makes it run, and what makes it fast

Three of the four production flags are not optional at seq 8192: without `nki-scan` the long convolution does not compile, without checkpointing the step does not fit in 24 GiB, and without per-block compile the graph exceeds the compiler's instruction limit.

LEVER	WHAT IT DOES	WORTH
`--hcl-kernel nki-scan`	The Hyena modal long convolution $h[s]=\Sigma_j r_j a_j^s$ is the transfer function of a first-order recurrence, so it becomes a scan. $C{\cdot}L^2 \rightarrow C{\cdot}S{\cdot}L$: 512× fewer MACs at 8192.	the only form that compiles at 8192
`--compile-blocks`	The whole-model graph is 13.57 M instructions against a 5 M cap (`NCC_EBVF030`). The cap is *per graph*, so compile eleven.	1.72× on the 8192 step
`--gradient-checkpointing`	Recompute 10 of 11 blocks. `use_reentrant=False` is mandatory — the reentrant path silently drops every LoRA gradient but the topmost.	makes 8192 fit; +3.0%, bit-identical
`--readout-block 10`	Build 11 of 32 blocks. The same cut applied to the checkpoint file is 13 GB → 4.46 GB.	two thirds of the weights skipped
op rewrites	`argmax` → cumsum/clamp/sum; `logaddexp` → max-shift; a window-spanning `Conv1d` → one `addmm`; RoPE `pow.Scalar` fixed in source.	get_fallback_ops() empty
fp32 depthwise conv in a bf16 model	Pinned deliberately: `F.conv1d` at `groups=4096, k=128` is 11–27× *slower* in bf16 than fp32 here.	measured, not cautionary

Two of those rewrites fix ops that fall back **silently** — the run completes with correct numbers and a host round-trip per step. That is why the census is a tool (`scripts/check_fallback_ops.py`) and not an eyeball: it clears tracking between stages, asserts the executed-op list is non-empty, and runs both dtypes, because the fallback set is dtype-dependent.

Artifact bundles

A bundle is one directory of tensors that pins everything a device port can get wrong. Dump one per device, then diff them.

FILE	CONTENTS	CATCHES
`inputs.pt`	the exact batches, replayed	a comparison run on the wrong input
`blocks0_10.pt`	the residual stream, per block	*where* the port diverges
`hidden0.pt`	the read-out the head consumes	*that* it diverged
`w0.pt`	trainable weights before step 0	the two arms started differently
`g1.pt`	gradients after the first backward	a wrong backward
`wN.pt`	weights after N steps	a wrong optimizer step

`hidden0` and `blocks0_10` are captured on the *first* forward, where `lora_B` is still zero and the adapters are exactly the identity. They are pure backbone activations — which is what lets a CUDA bundle produced by one task gate a port trained on another.

```
PYTHONPATH=src python scripts/dump_artifacts.py --device neuron --dtype bf16 \
    --inputs-from artifacts/cuda_seq200/inputs.pt --forward-only \
    --strict-determinism --out-dir artifacts/my_run

PYTHONPATH=src python scripts/dump_artifacts.py --compare \
    artifacts/cuda_seq200 artifacts/my_run
```

PIN THE WEIGHTS, DO NOT SEED THEM

`torch.manual_seed` feeds a *per-device* generator, so seeding alone gives CPU and Neuron unrelated head weights — measured rel_err 1.478, a comparison between two different models. `--init-from` refuses a partial pin rather than covering some tensors and leaving the rest random.

Things that fail without saying so

Each of these produces a run that completes and is wrong, or a job that is N× slower with no error. All are handled in the shipped code; they are here because you will meet them the moment you step outside it.

IF YOU…	WHAT HAPPENS
set the `TORCH_NEURONX_*` cache dirs after importing torch	no effect; the run uses `/tmp`, and a permission error there surfaces as a generic "Compilation error"
leave `NEURON_RT_VISIBLE_CORES=0` under torchrun	every rank would share one core — correct numbers, N× slower, no error. The driver refuses instead, naming the rank and the range.

IF YOU…	WHAT HAPPENS
leave the full range visible to every rank	each runtime claims all of it — `Failed to initialize Neuron Runtime: status code 1` on all but one rank, naming neither cause nor culprit. *Observed here.*
address the device by local rank after narrowing	`Device index 0 is out of range. Valid range is 3 to 3` — the visible set is narrowed, not renumbered. *Observed here.*
use `blocks[i] = torch.compile(...)`	`_orig_mod.` is inserted into 60 of 74 parameter names, breaking adapter checkpoints
leave `recompile_limit` at its default of 8	eleven blocks exceed it and dynamo falls back to eager, silently
read out via a forward hook on a compiled block	dynamo traces through the submodule and the hook never fires. *Observed here*; `compile_blocks()` now switches the read-out mode for you.
use reentrant activation checkpointing	a frozen backbone means block inputs do not require grad; every LoRA gradient but the topmost is dropped
skip `convert_wqkv_column_split` at load	q/k/v scrambled across heads, no error, read-out off by rel_err ~1.37
judge a gradient by magnitude alone	`lora_B` is zero-init, so an *absent* gradient reads as a small one
pipe a long device run through `tail`	it buffers until exit; a healthy compile looks hung

The last one has a corollary worth internalising: before believing any timing number, run `make health`. An orphaned process holding a NeuronCore fakes a mid-run throughput collapse, and a bimodal step-time trace is a contention signature until proven otherwise.

Apache-2.0. The model definition is re-implemented from ArcInstitute/evo2 and Zymrael/vortex, both Apache-2.0 — see `src/evo2_neuron/VENDORED.md` for source commits and the full patch list. Weights are Arc Institute's and are not redistributed. Numbers on this page were measured on a `trn2.48xlarge` with the AWS native-PyTorch beta and are reproducible with the commands shown.